

Stage 5 — Evals, Metrics, and Observability for Your Agent Platform

Goal: Turn your Stage 1–4 architecture into a **measurable, improvable system**. You will: (1) define quality metrics for your Creator Orchestrator, (2) implement an evaluation harness that replays traces and scores them, and (3) add basic observability (metrics + traces) — all backed by Postgres and standard telemetry, not ad-hoc prints.

By the end of Stage 5, you will have:

- An **offline eval harness** that can run your workflows on a test dataset and score them. [web:202][web:204][web:205][web:208]
- A simple **LLM-as-a-judge** evaluator for generation quality, plus a couple of rule-based metrics. [web:202][web:205][web:211]
- A minimal **observability layer** that tracks per-run latency, token usage, and error rates, with trace IDs tying everything together. [web:204][web:207][web:209][web:212]

This is how you move from "it works" to "we know how well it works and where it breaks." [web:202][web:204][web:206][web:210]

What You Already Have (Stage 1–4 Recap)

You now have:

- **Stage 1:** Single-agent loop with tools.
- **Stage 2:** Orchestrator + specialized agents.
- **Stage 3:** Memory layer in Postgres + pgvector (sessions, messages, memories, runs, events).
- **Stage 4:** Guardrails & policy layer with `guardrail_policies`, `guardrail_events`, and tool RBAC.

These give you **behavior, memory, and safety**, but not yet a systematic way to answer:

- "Is the system actually good at its job?" (quality)
- "Is it getting better or worse over time?" (regression)
- "What is it costing and how fast is it?" (performance)

These are the questions modern LLM evaluation practices focus on. [web:202][web:203][web:204][web:206][web:211]

Concept 1 — Evals vs. Observability

From eval/observability guides and modern platforms, think of two layers:[web:202][web:203][web:204][web:206][web:208][web:209][web:212]

1. Evals (Evaluation Harness)

- Run on **saved data** (offline) or recent production traces (online).[web:204][web:205][web:208]
- Score outputs and trajectories using metrics like:
 - Task completion.
 - Relevance to brief.
 - Factuality/groundedness.
 - Safety/compliance.
 - Style/brand fit.
- Use any combination of:
 - Rule-based metrics (exact match, regex, keyword checks).
 - Numeric metrics (BLEU/ROUGE/BERTScore for some tasks).[web:202][web:206]
 - LLM-as-a-judge metrics for subjective quality (clarity, helpfulness, tone).[web:202][web:205][web:211]
- Purpose: answer "how good" in a structured way.

2. Observability (Traces, Metrics, Logs)

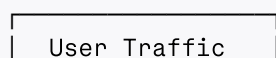
- Always-on instrumentation around the live system.[web:204][web:207][web:209][web:212]
- Focused on:
 - Latency (per run, per LLM call, per tool call).
 - Token usage and cost.[web:203][web:210]
 - Error rates and guardrail trigger rates.
- Uses: debugging, SLOs, cost control, detecting drift.[web:204][web:206][web:209]

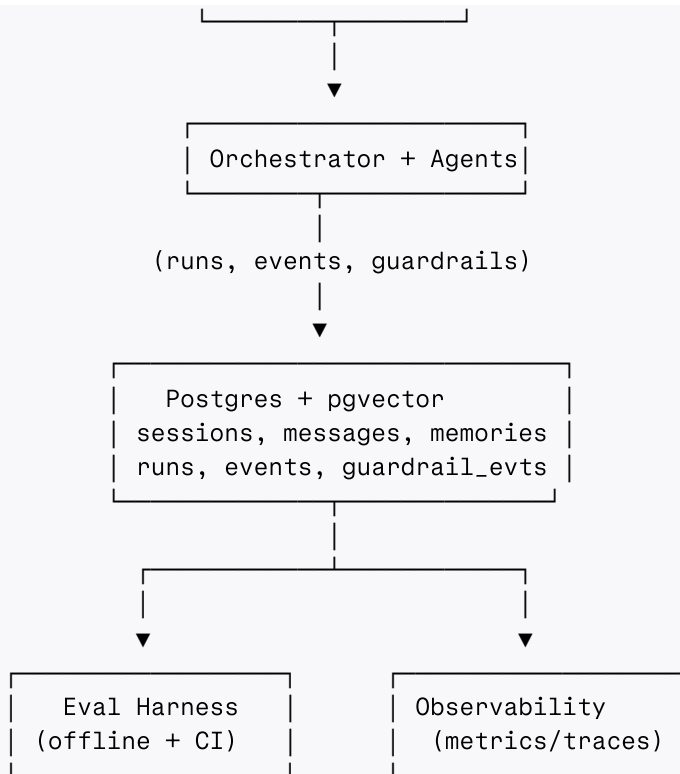
Traces are the shared foundation: a well-structured record of each run that both your offline eval harness and your live observability can read.[web:204][web:207][web:212]

You already store runs and events in Postgres; Stage 5 enriches those with metrics and adds a small eval harness over them.

High-Level Architecture with Evals & Observability

Extending your Stage 4 architecture:





- The **Eval Harness** reads from Postgres (or a CSV/JSON export of runs), replays or re-scores them, and writes scores back.
- Observability can push metrics/traces to Prometheus/OpenTelemetry, or stay in Postgres as a first step.[web:204][web:207][web:212]

Project Structure Additions

Building on Stage 4:

```
stage5-evals/
├── .env
├── config.py
├── requirements.txt
├── migrations/
│   └── 003_evals.sql
├── evals/
│   ├── __init__.py
│   ├── metrics.py      ← rule-based & numeric metrics
│   ├── judge.py        ← LLM-as-a-judge scoring
│   ├── harness.py      ← evaluation harness over runs/events
│   └── datasets.py     ← defining eval datasets / prompts
├── observability/
│   ├── __init__.py
│   ├── metrics.py      ← counters, timers, cost aggregation
│   └── tracing.py      ← trace/run IDs, span helpers (start/end)
├── agents/
├── orchestrator/
├── guardrails/
└── memory/
```

```
├─ tools/  
└─ main.py
```

You will:

1. Add an `eval_results` table for scores.
2. Instrument your code to capture **latency and token usage** per run.
3. Build a simple eval harness that scores runs and writes results back.

Step 1 — Eval Schema (Migration)

migrations/003_evals.sql

```
-- Stores evaluation results at run-level or step-level  
CREATE TABLE IF NOT EXISTS eval_results (  
  id          BIGSERIAL PRIMARY KEY,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  run_id      UUID REFERENCES runs(id) ON DELETE CASCADE,  
  session_id  UUID REFERENCES sessions(id) ON DELETE SET NULL,  
  evaluator   TEXT NOT NULL,          -- e.g. 'task_completion', 'brand_fit_judge'  
  scope       TEXT NOT NULL,          -- 'run', 'agent', 'message'  
  target      TEXT NOT NULL,          -- which agent or step (e.g. 'WriterAgent')  
  score       DOUBLE PRECISION,       -- numeric score 0-1 or 0-5  
  label       TEXT,                   -- optional label like 'pass'/'fail'  
  details     JSONB DEFAULT '{} '::jsonb  
);  
  
CREATE INDEX IF NOT EXISTS idx_eval_results_run ON eval_results(run_id);
```

This is your **canonical place** to store evaluation outcomes — one row per evaluator per run/agent/message.[\[web:208\]](#) You can later build dashboards or CI checks on top.

Apply:

```
psql "$DATABASE_URL" -f migrations/003_evals.sql
```

Step 2 — Minimal Observability Layer

Before building the eval harness, add some observability primitives that both live traffic and the eval harness can use.[\[web:204\]](#)[\[web:207\]](#)[\[web:209\]](#)[\[web:212\]](#)

observability/metrics.py

```
import time
from dataclasses import dataclass, field
from typing import Dict
from memory.db import get_cursor

@dataclass
class RunMetrics:
    latency_ms: float = 0.0
    token_prompt: int = 0
    token_completion: int = 0
    token_total: int = 0
    cost_usd: float = 0.0

def record_run_metrics(run_id: str, session_id: str | None, metrics: RunMetrics) -> None:
    """Persist run-level metrics in the runs table metadata."""
    with get_cursor() as cur:
        cur.execute(
            """
            UPDATE runs
            SET metadata = COALESCE(metadata, '{}':::jsonb) || %s::jsonb
            WHERE id = %s
            """,
            (
                {
                    "latency_ms": metrics.latency_ms,
                    "token_prompt": metrics.token_prompt,
                    "token_completion": metrics.token_completion,
                    "token_total": metrics.token_total,
                    "cost_usd": metrics.cost_usd,
                },
                run_id,
            ),
        )
```

You already have `runs.metadata`; this function just enriches it.

observability/tracing.py

```
import time
from contextlib import contextmanager
from typing import Generator, Optional
from observability.metrics import RunMetrics, record_run_metrics

@contextmanager
def run_timer(run_id: str, session_id: Optional[str] = None) -> Generator[RunMetrics, None, None]:
    """Context manager to time a full orchestrator run and collect metrics."""
    metrics = RunMetrics()
    start = time.perf_counter()
    try:
```

```

        yield metrics
    finally:
        end = time.perf_counter()
        metrics.latency_ms = (end - start) * 1000
        record_run_metrics(run_id, session_id, metrics)

```

You can now wrap your orchestrator run:

```

from observability.tracing import run_timer

run_id = episodic.start_run(...)

with run_timer(run_id, session_id) as rmetrics:
    result = orchestrator.run(...)
    # If you track token usage per LLM call, you can aggregate into rmetrics here.

episodic.finish_run(run_id, status="success")

```

Token usage and cost:

OpenAI responses include `usage` fields; aggregate those into `RunMetrics`. That gives you per-run latency and cost — core metrics recommended for production LLM apps.[\[web:203\]](#)[\[web:206\]](#)[\[web:209\]](#)

Step 3 — Defining Metrics for Your Creator Orchestrator

From modern evaluation guides, quality metrics should reflect both the *use case* (Creator Orchestrator) and the *system architecture* (multi-agent with memory and tools).[\[web:202\]](#)[\[web:203\]](#)[\[web:206\]](#)[\[web:211\]](#)

For your product, define at least these 5 metrics:

1. Task Completion (run-level)

- Does the orchestrator produce all required artifacts? (e.g., "5 clips, 2 posts, 1 email").
[\[web:203\]](#)[\[web:211\]](#)
- Metric: 1.0 if all present, 0.5 if partial, 0 if missing.

2. Brief Adherence / Relevance (writer-level)

- Does the written content follow the topic, target audience, and constraints in the brief?
[\[web:202\]](#)[\[web:203\]](#)[\[web:211\]](#)
- Metric: LLM-as-a-judge score 1–5.

3. Brand Fit / Tone (writer-level)

- Does the copy match brand voice? (e.g., "friendly, concise, no hype").[\[web:202\]](#)
[\[web:205\]](#)[\[web:206\]](#)
- Metric: LLM-as-a-judge score 1–5.

4. Safety / Compliance (run-level)

- Are there any policy/guardrail violations missed by Stage 4 guardrails?[web:202][web:203][web:206][web:211]
- Metric: 1.0 if safe, 0.0 if any violation.

5. Latency & Cost (run-level)

- Already measured via `RunMetrics`. Used for budgeting and SLOs.[web:203][web:206][web:209]

These metrics map directly onto the eval harness and observability primitives you're about to implement.

Step 4 — Rule-Based Metrics Implementation

`evals/metrics.py`

```
from typing import Dict, Any
from evals.store import save_eval_result

def metric_task_completion(run_id: str, session_id: str | None, result: Dict[str, Any]) -> float:
    """Check whether orchestrator returned all required artifacts."""
    # Assumes orchestrator returns a dict with keys like 'clips', 'posts', 'email'
    clips = result.get("clips") or []
    posts = result.get("posts") or []
    email = result.get("email")

    score = 1.0
    reasons = []

    if len(clips) < 5:
        score -= 0.5
        reasons.append(f"Expected >=5 clips, got {len(clips)}")
    if len(posts) < 2:
        score -= 0.25
        reasons.append(f"Expected >=2 posts, got {len(posts)}")
    if not email:
        score -= 0.25
        reasons.append("Missing email copy")

    if score < 0:
        score = 0.0

    save_eval_result(
        run_id=run_id,
        session_id=session_id,
        evaluator="task_completion",
        scope="run",
        target="orchestrator",
        score=score,
        label="pass" if score >= 0.75 else "fail",
    )
```

```
        details={"reasons": reasons},
    )
```

This uses a helper `save_eval_result` (next step) to write into `eval_results`.

Step 5 — LLM-as-a-Judge Evaluator

LLM-as-a-judge is now a standard recommendation for nuanced quality metrics (clarity, relevance, tone), especially for text content.[web:202][web:205][web:211]

`evals/judge.py`

```
from typing import Dict, Any
from openai import OpenAI
from config import OPENAI_API_KEY
from evals.store import save_eval_result

client = OpenAI(api_key=OPENAI_API_KEY)

JUDGE_PROMPT = """You are an expert marketing editor.

You will be given:
- A content brief.
- A generated marketing asset.

Evaluate the asset on a scale of 1 to 5 for:
- Relevance to the brief.
- Brand tone and style match.

Return a JSON object with this format:
{"relevance": <1-5>, "tone": <1-5>, "overall": <1-5>, "comments": "..."}
"""

def judge_brand_fit(run_id: str,
                    session_id: str | None,
                    brief: str,
                    content: str,
                    target: str) -> None:
    """Use an LLM judge to score relevance & tone for a piece of content."""
    messages = [
        {"role": "system", "content": JUDGE_PROMPT},
        {"role": "user", "content": f"BRIEF:\n{brief}\n\nCONTENT:\n{content}"},
    ]

    response = client.chat.completions.create(
        model="gpt-4o-mini", # cheap judge model
        messages=messages,
        response_format={"type": "json_object"},
    )
    import json
    data = json.loads(response.choices[0].message.content)
```



```

overall = float(data.get("overall", 0))

save_eval_result(
    run_id=run_id,
    session_id=session_id,
    evaluator="brand_fit_judge",
    scope="agent",
    target=target,
    score=overall,
    label="good" if overall >= 4 else "ok" if overall >= 3 else "bad",
    details=data,
)

```

This is a **simple but realistic** LLM-as-judge evaluator:

- Follows practices from LLM evaluation guides: clear rubric, numeric scores, JSON output, single-LLM judge.[web:202][web:205][web:211]
- Uses a cheaper model (gpt-4o-mini) as judge.
- Anchors results in `eval_results` for later aggregation.

In a more advanced setup, you would:

- Calibrate the judge against human labels on a small golden dataset.[web:205][web:211]
- Possibly use pairwise comparison or multiple judges.

Step 6 — Eval Store Helper

evals/store.py

```

from typing import Any, Optional
from memory.db import get_cursor

def save_eval_result(run_id: str,
                    session_id: Optional[str],
                    evaluator: str,
                    scope: str,
                    target: str,
                    score: float,
                    label: Optional[str] = None,
                    details: Optional[dict[str, Any]] = None) -> None:
    details = details or {}
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO eval_results (run_id, session_id, evaluator, scope, target, score, label, details)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """

```

```
(run_id, session_id, evaluator, scope, target, score, label, details),  
)
```

This is a simple write helper that everything else builds on.

Step 7 — Eval Datasets & Harness

From eval-harness patterns, a good harness must answer:[web:204][web:208]

- What requests to evaluate?
- Which evaluators to run?
- How to store the scores?

You will support **two modes**:

1. **Offline dataset eval** (CI / local): run orchestrator on curated prompts and score new runs.
2. **Production replay eval** (online): sample recent runs from Postgres and score them.

evals/datasets.py

```
from typing import List, Dict, Any  
  
def offline_eval_dataset() -> List[Dict[str, Any]]:  
    """Hard-coded small dataset for now. Later: load from JSON/CSV."""  
    return [  
        {  
            "topic": "Shorts: 60-second explainer on API rate limiting",  
            "brief": "Audience: junior backend devs. Tone: friendly, concise, no hype",  
        },  
        {  
            "topic": "YouTube: 8-minute video on Postgres + pgvector for RAG",  
            "brief": "Audience: indie hackers. Tone: practical, code-focused, minimal",  
        },  
    ]
```

evals/harness.py

```
from typing import List, Dict, Any  
from memory import short_term, episodic  
from observability.tracing import run_timer  
from evals.metrics import metric_task_completion  
from evals.judge import judge_brand_fit  
from evals.datasets import offline_eval_dataset  
  
class EvalHarness:  
    def __init__(self, orchestrator, user_id: str = "user:eval"):  
        self.orchestrator = orchestrator  
        self.user_id = user_id
```

```

def run_offline(self) -> None:
    """Run offline eval dataset through orchestrator and score each run."""
    for case in offline_eval_dataset():
        topic = case["topic"]
        brief = case["brief"]

        session_id = short_term.create_session(user_id=self.user_id, topic=topic)
        run_id = episodic.start_run(session_id, orchestrator="eval-offline",

        from observability.metrics import RunMetrics

        with run_timer(run_id, session_id) as rmetrics:
            # orchestrator.run should return a structured dict with all artifacts
            result = self.orchestrator.run(topic=topic, user_id=self.user_id, ses

        episodic.finish_run(run_id, status="success")

        # 1) Rule-based completion metric
        metric_task_completion(run_id, session_id, result)

        # 2) LLM-as-judge brand fit for posts (if any)
        posts = result.get("posts") or []
        for idx, post in enumerate(posts):
            judge_brand_fit(run_id, session_id, brief=brief, content=post, target

        print(f"Evaluated run {run_id} for topic: {topic}")

def run_on_existing_runs(self, run_ids: List[str]) -> None:
    """Re-score existing runs, assuming orchestrator saved outputs somewhere acco
    # For now, you can extend your orchestrator to store its structured results
    # in runs.metadata or a separate table and load them here.
    pass

```

This harness is intentionally minimal but **mirrors real-world evaluation systems**:[\[web:204\]](#)
[\[web:208\]](#)[\[web:210\]](#)

- It uses the same orchestrator and agents as production.
- It logs eval results to Postgres.
- It reuses tracing and run logging.

Later, you can embed this in CI or a nightly job to catch regressions.

Step 8 — Wiring Orchestrator for Evals

To support the eval harness, update your orchestrator's run signature so it can:

- Accept `session_id`, `run_id`, and `user_id`.
- Return a structured `result` dict that includes clips, posts, email, etc.

Example (simplified):

```

class CodeDrivenOrchestrator:
    def __init__(self, research_agent, writer_agent, reviewer_agent):
        self.research_agent = research_agent
        self.writer_agent = writer_agent
        self.reviewer_agent = reviewer_agent

    def run(self,
            topic: str,
            user_id: str | None = None,
            session_id: str | None = None,
            run_id: str | None = None,
            brief: str | None = None) -> dict:
        # assume session_id and run_id are created upstream (EvalHarness or main)

        # Step 1: Research
        research_text = self.research_agent.run(
            user_message=f"Research this topic: {topic}",
            session_id=session_id,
            run_id=run_id,
            user_id=user_id,
        )

        # Step 2: Writing (posts + email)
        posts = []
        email = None
        # ... call writer_agent with brief/research to generate assets ...

        # Step 3: Review
        # ... call reviewer_agent ...

        return {
            "topic": topic,
            "brief": brief,
            "research": research_text,
            "posts": posts,
            "email": email,
            # later: "clips": clips, etc.
        }

```

The important part is that **the orchestrator becomes a pure function from inputs to a structured output**, which the eval harness can consume repeatedly.

Step 9 — How to Run Stage 5 in Practice

1. Apply migration 003 for `eval_results`.
2. Instrument your orchestrator with `run_timer` and ensure it returns a structured result.
3. Implement eval modules (`evals/metrics.py`, `evals/judge.py`, `evals/store.py`, `evals/datasets.py`, `evals/harness.py`).
4. Add a CLI entry point, e.g. `python -m evals.harness_offline`, that:
 - Creates the orchestrator.

- Instantiates EvalHarness.
- Calls `run_offline()`.

5. After running offline evals, inspect Postgres:

- `SELECT * FROM eval_results ORDER BY created_at DESC LIMIT 20;`
- `SELECT id, metadata FROM runs ORDER BY created_at DESC LIMIT 10;` (to see latency and cost).

You should see:

- At least one `eval_results` row per run for `task_completion`.
- Several rows per run for `brand_fit_judge` when posts are generated.
- `runs.metadata` populated with `latency_ms` and token usage.

Step 10 — Enterprise Alignment & Next Steps

With Stage 5 implemented, your stack now has:

- **End-to-end evaluation harness** that can be used offline (CI) and later online on sampled production traces.[\[web:202\]](#)[\[web:204\]](#)[\[web:205\]](#)[\[web:208\]](#)
- **LLM-as-a-judge** evaluation of content quality, aligned with modern eval practices.[\[web:202\]](#)[\[web:205\]](#)[\[web:211\]](#)
- **Run-level metrics** for latency and cost persisted in Postgres, ready to be exported to Prometheus, OpenTelemetry, or Datadog later.[\[web:204\]](#)[\[web:207\]](#)[\[web:209\]](#)[\[web:214\]](#)
- **A unified data model:** `runs`, `events`, `guardrail_events`, and `eval_results` all keyed by `run_id`, which matches current recommendations for trace-based observability.[\[web:204\]](#)[\[web:207\]](#)[\[web:212\]](#)

This is exactly the kind of "evaluation harness + observability" pattern that recent industry guides recommend for production LLM and agent systems.[\[web:202\]](#)[\[web:204\]](#)[\[web:206\]](#)[\[web:208\]](#)[\[web:210\]](#)

When Stage 5 is "Done"

Check these boxes:

- ☐ You can run the offline eval harness and see new `runs`, `events`, and `eval_results` rows created.
- ☐ You can query `eval_results` to answer: "What is our average `brand_fit_judge` score over the last N runs?"
- ☐ You can see per-run latency and (optionally) token usage/cost in `runs.metadata`.
- ☐ You can explain the difference between:
 - Functional evals (`task_completion`),
 - Quality evals (`brand_fit_judge`),

- Observability metrics (latency, cost, error rates).

At that point, your Stage 1–5 architecture is not just **enterprise-style** from a code perspective — it has the **evaluation and observability spine** that real teams use to operate AI products in production.

From here, natural next steps are:

- Adding more evaluators (safety, hallucination, retrieval quality).
- Exporting metrics/traces to an external observability stack (OpenTelemetry + Grafana/Datadog).^{[web:207][web:209][web:214]}
- Building a small internal dashboard over `runs`, `guardrail_events`, and `eval_results` to monitor your Creator Orchestrator.